

```
alumno@iaw:~$ man ut1
```

UNIDAD DE TRABAJO 1 · RA1

Instalación de servidores de aplicaciones web

Ubuntu Server, Apache, MySQL y PHP: preparar, instalar, integrar, asegurar y documentar un servidor web

Índice

Introducción	3
Qué vas a aprender	3
Cómo usar este capítulo	3
1. Aplicaciones web y arquitectura cliente-servidor	5
1.1. Aplicaciones web y aplicaciones de escritorio	5
1.2. El modelo cliente-servidor	5
1.3. Páginas estáticas y dinámicas	5
1.4. Qué se ejecuta en el cliente y qué en el servidor	6
1.5. Arquitectura en niveles	6
2. El protocolo HTTP	7
2.1. Qué es HTTP	7
2.2. La petición	7
2.3. La respuesta	7
2.4. Las cabeceras	8
2.5. HTTPS	8
2.6. Herramientas para ver HTTP por dentro	9
3. El software de un servidor de aplicaciones web	10
3.1. Las piezas: la pila LAMP	10
3.2. Servidores web	10
3.3. Sistemas gestores de bases de datos	11
3.4. Lenguajes de servidor	11
3.5. Cómo se instala el software en Ubuntu	11
4. Análisis de requisitos y preparación del sistema	13
4.1. El análisis de requisitos	13
4.2. Virtualización	13
4.3. Ubuntu Server	14
4.4. Preparar el sistema antes de instalar	15
5. El servidor web Apache	17
5.1. Instalación y gestión del servicio	17
5.2. Los ficheros de Apache en Ubuntu	17
5.3. Sitios y hosts virtuales	18
5.4. Módulos	19
5.5. Comprobar la configuración y los registros	19
6. El sistema gestor de bases de datos MySQL	20
6.1. Instalación	20

6.2. El primer acceso	20
6.3. Asegurar la instalación	20
6.4. Bases de datos y usuarios	20
6.5. Configuración básica	21
6.6. Motores de almacenamiento	21
6.7. Herramientas gráficas	22
7. PHP y la integración de las piezas	23
7.1. Instalación	23
7.2. Cómo ejecuta Apache el código PHP	23
7.3. La configuración de PHP: php.ini	23
7.4. Extensiones	24
7.5. phpMyAdmin	24
7.6. Comprobar que todo está integrado	24
8. Seguridad en los accesos al servidor	26
8.1. Tres principios	26
8.2. Usuarios y permisos del sistema	26
8.3. Apache: no dar información de más	26
8.4. MySQL: nadie entra por donde no debe	27
8.5. El cortafuegos: ufw	27
8.6. SSH	27
8.7. HTTPS	28
8.8. Actualizaciones	28
8.9. Verificar la seguridad	28
9. Comprobación del sistema	30
9.1. Problemas frecuentes	30
10. Plataformas integradas	32
10.1. Qué son	32
10.2. Ventajas e inconvenientes	32
10.3. XAMPP	32
10.4. La instalación manual en Windows	33
11. Documentación de la instalación	34
11.1. Por qué documentar	34
11.2. Qué documentar	34
11.3. Cómo documentar	34
Ideas clave de la unidad	36
Glosario	37
Autoevaluación	38

Introducción

Una aplicación web no funciona sola. Detrás de cada página que se abre en un navegador hay un **servidor**: un ordenador con un sistema operativo, un programa que atiende las peticiones, un lenguaje que genera las páginas y una base de datos que guarda la información. Montar, configurar, asegurar y documentar ese servidor es trabajo de un administrador de sistemas, y es lo que aprenderás en esta unidad.

Al terminarla serás capaz de preparar desde cero un servidor con **Ubuntu Server, Apache, MySQL y PHP**, comprobar que sus piezas se comunican entre sí, protegerlo frente a accesos indebidos y dejar documentado todo lo que has hecho, para que otra persona, o tú dentro de un mes, pueda entenderlo y repetirlo.

Qué vas a aprender

Esta unidad desarrolla el **resultado de aprendizaje 1 (RA1)** del módulo: *prepara el entorno de desarrollo y los servidores de aplicaciones web instalando e integrando las funcionalidades necesarias*. Se evalúa con estos criterios, y al lado tienes el apartado del capítulo donde se trabaja cada uno:

Criterio de evaluación	Apartados
a) Se ha identificado el software necesario para su funcionamiento	3, 4
b) Se han identificado las diferentes tecnologías empleadas	1, 2, 3
c) Se han instalado y configurado servidores web y de bases de datos	5, 6
d) Se han reconocido las posibilidades de procesamiento en los entornos cliente y servidor	1
e) Se han añadido y configurado los componentes y módulos necesarios para el procesamiento de código en el servidor	7
f) Se ha instalado y configurado el acceso a bases de datos	6, 7
g) Se ha establecido y verificado la seguridad en los accesos al servidor	8
h) Se han utilizado plataformas integradas orientadas a la prueba y desarrollo de aplicaciones web	10
i) Se han documentado los procedimientos realizados	11

Cómo usar este capítulo

El capítulo sigue el orden en que se monta un servidor de verdad: primero se entiende qué es una aplicación web y cómo se comunica (apartados 1 a 3), después se prepara el sistema (4), se instalan sus piezas (5 a 7), se aseguran (8), se comprueba que todo funciona (9) y se documenta (11). El apartado 10 presenta las plataformas integradas, que montan todo lo anterior de una vez para trabajar en local.

A lo largo del texto encontrarás tres tipos de recuadros:

Idea clave. Lo que tienes que llevarte de cada apartado.

Cuidado. Errores habituales y cosas que conviene no hacer.

En el aula. Cómo se aplica en las prácticas y en el proyecto de la unidad.

Los comandos están pensados para **Ubuntu Server LTS** y se escriben en la terminal tal cual aparecen. Las líneas que empiezan por `$` son órdenes; las demás, lo que responde el sistema.

1. Aplicaciones web y arquitectura cliente-servidor

1.1. Aplicaciones web y aplicaciones de escritorio

Una **aplicación de escritorio** se instala y se ejecuta en el propio ordenador del usuario: un procesador de textos, un editor de imágenes. Una **aplicación web** se usa a través de un navegador, sin instalar nada: el aula virtual, el correo web o una tienda en línea. La mayor parte de su trabajo ocurre en un servidor remoto, y el navegador se limita a pedir y a mostrar.

Esta forma de trabajar tiene ventajas claras: la aplicación funciona igual en cualquier sistema operativo con navegador, se actualiza una sola vez en el servidor para todos los usuarios y los datos están centralizados. A cambio, depende de la red, y el servidor tiene que estar siempre disponible y bien protegido, porque atiende a todo el mundo a la vez.

1.2. El modelo cliente-servidor

Las aplicaciones web siguen el **modelo cliente-servidor**. El **cliente** es quien pide un servicio: en la web, el navegador. El **servidor** es quien lo presta: un programa que espera peticiones y las responde. La palabra *servidor* se usa para las dos cosas, el programa (el servidor web Apache) y la máquina donde se ejecuta (el ordenador del centro de datos); el contexto aclara a cuál nos referimos.

Cuando escribes una dirección en el navegador ocurren, simplificando, estos pasos:

1. **Resolución del nombre.** El navegador pregunta a un servidor DNS qué dirección IP corresponde al nombre escrito, por ejemplo `www.ejemplo.es`.
2. **Petición.** El navegador se conecta a esa IP y envía una petición HTTP pidiendo una página.
3. **Trabajo del servidor.** El servidor web localiza lo que se pide. Si es una página dinámica, ejecuta el código correspondiente, que puede consultar una base de datos, y genera el HTML resultante.
4. **Respuesta.** El servidor devuelve el documento generado, o un código de error si no ha podido servirlo.
5. **Presentación.** El navegador interpreta el HTML, aplica las hojas de estilo CSS, ejecuta el JavaScript y muestra la página.

Los pasos 2 y 5 ocurren en el cliente; los pasos 3 y 4, en el servidor.

1.3. Páginas estáticas y dinámicas

Una **página estática** es un fichero HTML guardado en el servidor, que se envía tal cual cada vez que alguien la pide. Todos los visitantes ven exactamente lo mismo, y para cambiarla hay que editar el fichero. Es suficiente para la página de presentación de una empresa o un currículum.

Una **página dinámica** no existe como fichero terminado: el servidor la **genera en el momento** de cada petición, ejecutando un programa que combina una plantilla con datos. Así, cada usuario puede ver

algo distinto: su área personal, sus calificaciones o el contenido de su carrito. Casi toda la web actual es dinámica, y este módulo trata de montar y mantener los servidores capaces de generarla.

1.4. Qué se ejecuta en el cliente y qué en el servidor

En una aplicación web conviven dos tipos de código, que se ejecutan en sitios distintos:

	En el cliente (navegador)	En el servidor
Lenguajes	HTML, CSS y JavaScript	PHP, Python, Java, JavaScript con Node.js...
Para qué se usa	Presentación, efectos visuales, interactividad y avisos rápidos al usuario	Acceso a bases de datos, gestión de usuarios y contraseñas, lógica de negocio y cálculos que importan
Quién ve el código	Cualquiera: el navegador lo recibe y el usuario puede leerlo y modificarlo	Nadie: el código se ejecuta en el servidor y el usuario solo recibe el resultado
Ventaja principal	Respuesta inmediata, sin esperar al servidor, y menos carga para este	Acceso a los datos y a los recursos del servidor, con control total sobre lo que se hace

Una forma sencilla de comprobarlo: en cualquier web, abre el menú contextual y elige *Ver código fuente*. Verás HTML, CSS y JavaScript, pero nunca PHP, porque el PHP ya se ejecutó en el servidor antes de enviar la página.

La regla de oro. Todo lo que se comprueba en el navegador se puede saltar, porque el usuario controla su navegador. Validar datos en el cliente es una cuestión de **comodidad** (avisar rápido de un error); validarlos en el servidor es una cuestión de **seguridad**. Precios, permisos, contraseñas y cualquier dato que vaya a guardarse se comprueban siempre en el servidor.

Un ejemplo: una tienda que calcula el precio total con JavaScript y se lo envía al servidor para cobrarlo. Si el servidor se fía de ese precio, un usuario con conocimientos puede modificarlo en su navegador y pagar lo que quiera. El total que se cobra tiene que calcularlo el servidor con sus propios datos.

1.5. Arquitectura en niveles

Las aplicaciones web se organizan en capas o **niveles**, cada una con una función:

- **Presentación:** lo que ve el usuario. Se genera en el servidor y se muestra en el navegador.
- **Lógica:** el código que procesa las peticiones y decide qué hacer. En nuestro caso, PHP.
- **Datos:** donde se guarda la información. En nuestro caso, MySQL.

En una instalación pequeña, como la de este módulo, las tres capas conviven en la misma máquina. En instalaciones grandes se reparten en máquinas distintas (arquitectura en tres niveles): así cada una se puede ampliar por separado, y la base de datos queda más protegida, sin acceso directo desde internet.

2. El protocolo HTTP

2.1. Qué es HTTP

HTTP (*Hypertext Transfer Protocol*) es el protocolo con el que se comunican el navegador y el servidor web: el idioma y las reglas de la conversación. Es un protocolo de **petición y respuesta**: el cliente siempre pregunta primero y el servidor responde. También es **sin estado**: cada petición es independiente, y el servidor no recuerda por sí mismo las anteriores. Para que una aplicación recuerde quién eres entre una página y otra hacen falta mecanismos adicionales, como las cookies y las sesiones, que veremos en la unidad de PHP.

Por defecto, un servidor web escucha las peticiones HTTP en el **puerto 80** y las HTTPS en el **puerto 443**.

2.2. La petición

Una petición HTTP tiene una línea inicial, unas cabeceras y, a veces, un cuerpo:

```
GET /index.html HTTP/1.1
Host: www.ejemplo.es
User-Agent: Mozilla/5.0 (X11; Linux x86_64) Firefox
Accept: text/html
```

La primera línea indica el **método**, el **recurso** que se pide y la versión del protocolo. Los métodos más habituales son:

Método	Para qué se usa
GET	Pedir un recurso. Es lo que hace el navegador al abrir una página o seguir un enlace
POST	Enviar datos al servidor, por ejemplo al enviar un formulario
HEAD	Como GET, pero pidiendo solo las cabeceras de la respuesta, sin el contenido

La cabecera `Host` indica a qué sitio web va dirigida la petición. Es imprescindible, porque un mismo servidor puede alojar muchos sitios distintos con la misma IP (lo veremos al hablar de los hosts virtuales).

2.3. La respuesta

La respuesta tiene una línea de estado, unas cabeceras, una línea en blanco y el contenido:


```

HTTP/1.1 200 OK
Server: Apache
Content-Type: text/html; charset=UTF-8
Content-Length: 10671

<!DOCTYPE html>
<html> ... </html>

```

El número de la línea de estado es el **código de estado**, que resume cómo ha ido la petición. Su primera cifra indica la familia:

Familia	Significado	Códigos habituales
1xx	Información: la petición se está procesando	Poco frecuentes
2xx	Éxito	200 OK
3xx	Redirección: el recurso está en otra dirección	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Error del cliente: la petición no es correcta o no está permitida	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Error del servidor: la petición era correcta, pero el servidor ha fallado	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

El primer número dice de quién es el problema. Un 4xx apunta al cliente: una dirección mal escrita, un recurso que no existe o sin permiso. Un 5xx apunta al servidor: un error en el código PHP, un servicio caído o una mala configuración. Como administrador, los 5xx son tu responsabilidad.

2.4. Las cabeceras

Las cabeceras son pares `nombre: valor` que añaden información a la petición o a la respuesta. En la respuesta, las más útiles para un administrador son:

- **Server:** el software que ha respondido, a veces con su versión y su sistema operativo. Muchos servidores la recortan u ocultan, porque dar esa información facilita el trabajo a un atacante (apartado 8.3).
- **Content-Type:** el tipo de contenido que se envía, por ejemplo `text/html` para una página o `image/png` para una imagen, y su codificación de caracteres.
- **Location:** en las redirecciones (3xx), la dirección a la que debe ir el navegador.
- **Set-Cookie:** una cookie que el navegador debe guardar y devolver en las siguientes peticiones.

2.5. HTTPS

HTTPS es HTTP sobre una conexión cifrada con **TLS**. El contenido de la conversación es el mismo, pero nadie que esté entre el cliente y el servidor puede leerlo ni modificarlo, y el navegador puede comprobar, mediante un **certificado digital**, que habla con el servidor auténtico.

Hoy HTTPS es la norma: los navegadores marcan como no seguras las páginas HTTP, y la mayoría de los servidores responden a una petición HTTP con una redirección **301** hacia la misma dirección con HTTPS.

2.6. Herramientas para ver HTTP por dentro

- **curl** hace peticiones HTTP desde la terminal. Con la opción **-I** hace una petición HEAD y muestra solo las cabeceras de la respuesta:

```
$ curl -I http://www.ejemplo.es
HTTP/1.1 301 Moved Permanently
Location: https://www.ejemplo.es/
Server: nginx
Content-Type: text/html
```

- **Las herramientas de desarrollo del navegador** (tecla F12, pestaña *Red*) muestran cada petición que hace una página, con su método, su código de estado, sus cabeceras y el tiempo que ha tardado.

En el aula. El ejercicio 2 del primer día consiste en lanzar `curl -I` contra dos webs reales y leer su código de estado y sus cabeceras. Cuando tu servidor esté montado, `curl -I localhost` será tu primera comprobación de que Apache responde.

3. El software de un servidor de aplicaciones web

3.1. Las piezas: la pila LAMP

Un servidor de aplicaciones web necesita cuatro piezas, que forman lo que se llama una **pila** (*stack*). La más extendida es **LAMP**:

Letra	Pieza	Función
L	Linux	El sistema operativo sobre el que funciona todo
A	Apache	El servidor web: recibe las peticiones HTTP y devuelve las respuestas
M	MySQL	El sistema gestor de bases de datos (SGBD): guarda y sirve los datos
P	PHP	El lenguaje de programación del servidor: genera las páginas dinámicas

Relacionando las piezas con los pasos del apartado 1.2: Apache recibe la petición y entrega la respuesta; PHP ejecuta el código que genera la página, y MySQL le proporciona los datos.

Existen variantes con otras piezas que cumplen el mismo papel:

- **LEMP**: cambia Apache por **Nginx** (se pronuncia *engine-x*, de ahí la E).
- **WAMP** y **MAMP**: la misma pila sobre Windows o macOS.
- **MariaDB** en lugar de MySQL: una derivación de MySQL creada por sus desarrolladores originales, compatible con él en el uso habitual. Muchas distribuciones y proveedores de alojamiento la usan por defecto.

3.2. Servidores web

Servidor	Características
Apache HTTP Server	Libre, multiplataforma y modular: su funcionalidad se amplía activando módulos. Muy documentado y con una enorme base de usuarios. Permite configuración por directorio con ficheros <code>.htaccess</code> , algo que usan muchas aplicaciones PHP
Nginx	Libre y muy eficiente con muchas conexiones simultáneas. Se usa mucho como servidor de contenido estático y como proxy inverso delante de otros servidores
IIS	El servidor web de Microsoft, integrado en Windows Server

En este módulo usamos **Apache** porque es el más habitual con PHP, porque la mayoría de gestores de contenidos (WordPress, Joomla, Moodle) están pensados para él y porque su organización en módulos ayuda a entender cómo se integran las piezas.

3.3. Sistemas gestores de bases de datos

Un **SGBD** es el software que crea, almacena y gestiona las bases de datos, y que hace de intermediario entre los datos y los programas que los usan. En entornos web los más utilizados son:

- **MySQL**: el más extendido en aplicaciones web con PHP, hoy propiedad de Oracle, con una edición comunitaria libre.
- **MariaDB**: la alternativa comunitaria a MySQL, compatible con él.
- **PostgreSQL**: libre, muy completo y riguroso con los estándares; frecuente en aplicaciones más complejas.
- **SQLite**: una base de datos en un único fichero, sin servidor, útil para aplicaciones pequeñas.

Todos ellos son **relacionales**: guardan los datos en tablas y se consultan con SQL. Existen también bases de datos **NoSQL**, como MongoDB, que guardan documentos en lugar de filas; son otra familia, con otros usos.

3.4. Lenguajes de servidor

Lenguaje	Uso en la web
PHP	Nacido para la web y el más usado en el lado del servidor. En él están hechos WordPress, Joomla, Moodle o Nextcloud
Python	Con entornos como Django o Flask
JavaScript (Node.js)	El mismo lenguaje del navegador, ejecutado en el servidor
Java	Frecuente en aplicaciones empresariales grandes
C# (ASP.NET)	La plataforma de Microsoft

Usamos **PHP** porque todos los gestores de contenidos y aplicaciones que instalaremos en el módulo están escritos en él.

3.5. Cómo se instala el software en Ubuntu

En Ubuntu el software se instala en forma de **paquetes** con el gestor **apt**. Los paquetes se descargan de los **repositorios**: servidores de Canonical, y copias suyas llamadas espejos o *mirrors*, con miles de programas preparados y probados para cada versión de Ubuntu.

Orden	Qué hace
<code>sudo apt update</code>	Descarga la lista actualizada de paquetes disponibles. No instala nada
<code>sudo apt upgrade</code>	Instala las nuevas versiones de los paquetes ya instalados
<code>sudo apt install paquete</code>	Instala un paquete y todo lo que necesita (sus dependencias)
<code>sudo apt remove paquete</code>	Desinstala un paquete, conservando su configuración
<code>apt policy paquete</code>	Muestra la versión instalada y la versión candidata a instalarse

Orden	Qué hace
<code>apt show paquete</code>	Muestra la descripción y los datos de un paquete

Los paquetes que forman un servidor LAMP en Ubuntu son estos:

Paquete	Qué es
<code>apache2</code>	El servidor web Apache
<code>mysql-server</code>	El servidor de bases de datos MySQL
<code>php</code>	El intérprete de PHP
<code>libapache2-mod-php</code>	El módulo que permite a Apache ejecutar código PHP
<code>php-mysql</code>	La extensión que permite a PHP conectarse a MySQL

Los dos olvidos típicos. Sin `libapache2-mod-php`, Apache no sabe qué hacer con un fichero `.php` y lo envía como texto: el navegador muestra el código o lo descarga. Sin `php-mysql`, PHP no puede conectar con la base de datos y da un error del tipo *could not find driver*.

Ubuntu tiene además otro sistema de paquetes, los **snaps**: aplicaciones autocontenidas, con todo lo que necesitan dentro, que se actualizan solas. En los servidores de este módulo instalamos siempre con `apt`.

4. Análisis de requisitos y preparación del sistema

4.1. El análisis de requisitos

Antes de instalar nada hay que saber **qué debe hacer el sistema y en qué condiciones**. Es el análisis de requisitos: fijar las condiciones de partida para no descubrir a mitad del trabajo que el servidor se queda corto o que una pieza no es compatible con otra. Se trata de definir **qué** debe hacer el sistema, no **cómo** se va a conseguir.

Los requisitos suelen agruparse así:

Tipo	Qué especifica	Ejemplos
Funcionales	Lo que el sistema debe ser capaz de hacer	Servir la web de la empresa, permitir que los clientes se registren, guardar pedidos
No funcionales	Cómo debe hacerlo	Rendimiento, disponibilidad, seguridad, facilidad de mantenimiento
Otros	Condiciones externas	Compatibilidad con el hardware y el software existentes, legislación aplicable (por ejemplo, la de protección de datos)

De ahí salen las decisiones técnicas: qué sistema operativo, qué versiones de cada pieza, cuánta memoria y disco, y si todo irá en una máquina o en varias. Un punto clave es la **compatibilidad de versiones**: una aplicación puede exigir una versión mínima de PHP o de MySQL, y el sistema operativo determina qué versiones hay disponibles en sus repositorios.

4.2. Virtualización

Hoy la mayoría de los servidores son **máquinas virtuales**: ordenadores simulados por software, que funcionan dentro de un ordenador físico junto a otros.

Capa	Qué es
Anfitrión (<i>host</i>)	El ordenador físico y su sistema operativo
Hipervisor	El software que crea y ejecuta las máquinas virtuales
Máquina virtual	Un ordenador simulado, con su CPU, su RAM, su disco y su tarjeta de red virtuales
Invitado (<i>guest</i>)	El sistema operativo instalado dentro de la máquina virtual

Hay dos tipos de hipervisores. Los de **tipo 1** se instalan directamente sobre el hardware, sin sistema operativo debajo, y son los que se usan en los centros de datos (VMware ESXi, Proxmox, Hyper-V). Los de **tipo 2** funcionan como un programa más encima de un sistema operativo, y son los habituales en un puesto de trabajo (VirtualBox, VMware Workstation).

Las ventajas de trabajar con máquinas virtuales son muchas: se pueden crear y destruir en minutos, cada una está aislada de las demás, se pueden copiar y trasladar, y se pueden guardar **instantáneas** (*snapshots*) para volver a un estado anterior si algo sale mal.

Los modos de red en VirtualBox

Cómo se conecta la máquina virtual a la red decide quién puede llegar hasta ella:

Modo	Cómo funciona	Uso típico
NAT (por defecto)	La máquina sale a internet a través del anfitrión, que la oculta. Recibe siempre una IP interna (10.0.2.15)	Navegar e instalar paquetes. Desde fuera no se puede llegar a ella
Adaptador puente	La máquina se conecta a la red real como un equipo más, y el router le asigna su propia IP	Servidores: los demás equipos de la red pueden acceder a ella
Red NAT	Varias máquinas virtuales comparten una red NAT y se ven entre sí	Laboratorios con varias máquinas
Solo anfitrión	Red privada entre el anfitrión y sus máquinas, sin salida a internet	Pruebas aisladas

Para un servidor web usamos **adaptador puente**, porque tiene que ser accesible desde otros equipos. Si la red no lo permite, la alternativa es NAT con **redirección de puertos**: el anfitrión reenvía a la máquina virtual lo que llega a uno de sus puertos (por ejemplo, el 8080 del anfitrión al 80 de la máquina).

4.3. Ubuntu Server

Usamos **Ubuntu Server** por varios motivos: es libre, está muy extendido en servidores y en la nube, tiene una documentación y una comunidad enormes, y sus versiones **LTS** (*Long Term Support*) reciben cinco años de actualizaciones de seguridad gratuitas. En un servidor se busca estabilidad, no la última novedad.

La edición de servidor **no tiene entorno gráfico**: se administra por terminal. Así consume menos memoria y tiene menos programas que mantener y que puedan tener fallos de seguridad. En la práctica, además, los servidores se administran a distancia, por SSH.

Durante la instalación se toman decisiones que conviene conocer y documentar:

Decisión	Qué implica
Idioma y teclado	La distribución de teclado equivocada cambia los símbolos, algo crítico en una terminal
Tipo de instalación	La normal incluye las herramientas básicas de administración; la minimizada las recorta
Red	Por defecto, IP automática por DHCP . En un servidor definitivo se configura una IP fija
Proxy y espejo	Cómo sale el sistema a internet y de qué servidor descargará los paquetes
Disco	Usar el disco entero con LVM (gestor de volúmenes lógicos), que permite ampliar el espacio más adelante sin reinstalar
Perfil	El nombre del servidor (<i>hostname</i>) y el usuario administrador

Decisión	Qué implica
SSH	Instalar OpenSSH para administrar el servidor a distancia
Snaps	Aplicaciones adicionales que se pueden instalar durante la instalación. En nuestro caso, ninguna

El **nombre del servidor** identifica la máquina en la red y aparece en el *prompt* de la terminal (usuario@nombre:~\$). Solo admite letras minúsculas, números y guiones.

Ubuntu **no permite entrar directamente como root**, el superusuario. El usuario creado en la instalación trabaja con permisos normales y, cuando necesita hacer tareas de administración, las ejecuta con **sudo**, que pide su propia contraseña y deja registro de lo que se hace. Es una aplicación del **principio de mínimo privilegio**: trabajar siempre con los permisos justos, y elevarlos solo cuando hace falta.

4.4. Preparar el sistema antes de instalar

Con el sistema operativo instalado, antes de montar el servidor web conviene dejar la máquina en buen estado:

Actualizar el sistema, para partir de las últimas correcciones de seguridad:

```
sudo apt update
sudo apt upgrade
```

Comprobar la versión y el nombre del sistema, los primeros datos de la documentación:

```
lsb_release -a
hostnamectl
```

Comprobar la red y la IP de la máquina:

```
ip a
ping -c 3 8.8.8.8
```

La orden `ip a` muestra las interfaces de red. `lo` es la interfaz de bucle local (127.0.0.1), con la que la máquina se comunica consigo misma; `enp0s3` suele ser la tarjeta de red de la máquina virtual. La IP aparece en la línea `inet`. Las direcciones que empiezan por `10.`, `172.16.` a `172.31.` y `192.168.` son **privadas**: solo son válidas dentro de una red local.

La orden `ping` envía mensajes ICMP a otra máquina y espera respuesta. La dirección `8.8.8.8` corresponde a un servidor DNS público de Google, que se usa como destino de prueba porque siempre responde: si llegan las respuestas, el servidor tiene salida a internet.

Comprobar la hora, importante para que los registros y los certificados sean coherentes:

```
timedatectl
```


Comprobar que no hay otro servicio ocupando los puertos que va a usar el servidor web (80 y 443).

En un equipo solo un programa puede escuchar en un puerto a la vez:

```
sudo ss -tlnp
```

Hacer una copia de seguridad si la máquina ya tenía datos, guardándola fuera de ella. En una máquina virtual, lo más cómodo es tomar una **instantánea** antes de cada cambio importante.

IP fija

Un servidor definitivo debe tener una **IP fija**, porque su dirección no puede cambiar cada vez que se reinicia. En Ubuntu Server la red se configura con **netplan**, mediante un fichero YAML en

/etc/netplan/. Un ejemplo:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses: [192.168.1.50/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Los cambios se prueban con `sudo netplan try`, que los deshace solo si no se confirman, y se aplican con `sudo netplan apply`.

No pongas una IP fija sin consultarlo. En una red compartida, como la del aula, una IP fija elegida al azar puede coincidir con la de otro equipo y dejar a los dos sin red. En las prácticas trabajamos con DHCP, y por eso **al empezar cada sesión hay que mirar la IP** con `ip a`: puede haber cambiado.

5. El servidor web Apache

5.1. Instalación y gestión del servicio

Apache se instala con un solo paquete:

```
sudo apt install apache2
```

Al terminar, el servicio queda en marcha y configurado para arrancar con el sistema. Los servicios de Ubuntu se gestionan con **systemctl**:

Orden	Qué hace
<code>systemctl status apache2</code>	Muestra si el servicio está activo y sus últimos mensajes
<code>sudo systemctl start apache2</code>	Arranca el servicio
<code>sudo systemctl stop apache2</code>	Detiene el servicio
<code>sudo systemctl restart apache2</code>	Lo detiene y lo vuelve a arrancar
<code>sudo systemctl reload apache2</code>	Relee la configuración sin cortar las conexiones en curso
<code>sudo systemctl enable apache2</code>	Hace que arranque al iniciar el sistema

Para comprobar que responde, desde el propio servidor:

```
$ curl -I localhost
HTTP/1.1 200 OK
Server: Apache/2.4.x (Ubuntu)
Content-Type: text/html
```

Y desde otro equipo de la red, abriendo en el navegador `http://` seguido de la IP del servidor, debe aparecer la página por defecto de Apache en Ubuntu, con el título *It works!*

5.2. Los ficheros de Apache en Ubuntu

Ubuntu organiza la configuración de Apache en varios ficheros y carpetas, dentro de `/etc/apache2/`:

Ruta	Contenido
<code>apache2.conf</code>	La configuración principal. Rara vez hace falta tocarla
<code>ports.conf</code>	Los puertos en los que escucha Apache (80 y 443)
<code>sites-available/</code>	Un fichero por cada sitio web configurado
<code>sites-enabled/</code>	Los sitios activos: enlaces a los ficheros de <code>sites-available/</code>
<code>mods-available/</code> y <code>mods-enabled/</code>	Los módulos disponibles y los activos
<code>conf-available/</code> y <code>conf-enabled/</code>	Fragmentos de configuración general, como <code>security.conf</code>

Ruta	Contenido
<code>envvars</code>	Variables de entorno, como el usuario con el que se ejecuta Apache

Otras rutas importantes:

Ruta	Contenido
<code>/var/www/html/</code>	La carpeta raíz del sitio por defecto (<i>DocumentRoot</i>): lo que se publica
<code>/var/log/apache2/access.log</code>	El registro de cada petición recibida
<code>/var/log/apache2/error.log</code>	El registro de errores

Apache se ejecuta con el usuario **www-data**, un usuario sin privilegios creado para él. Arranca como root para poder abrir el puerto 80, reservado a administradores, y después cede el trabajo a procesos de www-data. Así, si alguien aprovechara un fallo de Apache, solo tendría los permisos de www-data, no los de root.

5.3. Sitios y hosts virtuales

Un mismo servidor Apache puede alojar muchos sitios web distintos. Cada uno se define en un **host virtual** (*VirtualHost*), y Apache sabe a cuál va dirigida cada petición por la cabecera `Host` (apartado 2.2). Así funcionan las empresas de alojamiento, que tienen cientos de webs en una sola máquina.

Un host virtual se define en un fichero de `sites-available/`. Por ejemplo, `/etc/apache2/sites-available/clinica.conf`:

```
<VirtualHost *:80>
    ServerName clinica.local
    DocumentRoot /var/www/clinica
    ErrorLog ${APACHE_LOG_DIR}/clinica_error.log
    CustomLog ${APACHE_LOG_DIR}/clinica_access.log combined
</VirtualHost>
```

Y se activa, se comprueba y se recarga la configuración:

```
sudo a2ensite clinica.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

`a2ensite` crea el enlace en `sites-enabled/`, y `a2dissite` lo quita. Para que el nombre `clinica.local` lleve al servidor sin tener un DNS propio, se añade al fichero `hosts` del equipo cliente (`/etc/hosts` en Linux, `C:\Windows\System32\drivers\etc\hosts` en Windows) una línea con la IP del servidor y el nombre:

```
192.168.1.50  clinica.local
```

5.4. Módulos

Apache es **modular**: el núcleo es pequeño y cada funcionalidad se añade con un módulo. Algunos ejemplos son `mod_rewrite` (reescritura de direcciones, que usan muchos gestores de contenidos), `mod_ssl` (HTTPS) o el módulo de PHP. Se activan y desactivan con:

```
sudo a2enmod rewrite
sudo a2dismod autoindex
sudo systemctl restart apache2
```

Y se listan los módulos cargados con `apache2ctl -M`.

5.5. Comprobar la configuración y los registros

Antes de recargar Apache tras un cambio, siempre hay que comprobar que la configuración no tiene errores:

```
$ sudo apache2ctl configtest
Syntax OK
```

Si Apache no arranca, los motivos más habituales son un error de sintaxis en la configuración o un puerto ya ocupado por otro programa. Para averiguarlo:

```
sudo journalctl -u apache2 --no-pager | tail -20
sudo tail -f /var/log/apache2/error.log
```

La orden `tail -f` deja el registro abierto y muestra cada nueva línea en el momento en que se escribe: muy útil para ver qué ocurre al recargar una página.

El ciclo de trabajo con Apache. Editar la configuración, comprobarla con `apache2ctl configtest`, recargar con `systemctl reload apache2` y, si algo falla, mirar los registros. Nunca recargar sin comprobar antes.

6. El sistema gestor de bases de datos MySQL

6.1. Instalación

```
sudo apt install mysql-server
```

Como Apache, el servicio queda en marcha y activado al arrancar, y se gestiona con `systemctl` (el servicio se llama `mysql`). MySQL escucha por defecto en el **puerto 3306**.

6.2. El primer acceso

En Ubuntu, el usuario **root de MySQL** (que no es el mismo que el root del sistema) se autentica por defecto con el complemento **auth_socket**: no usa contraseña, sino que comprueba que quien se conecta es el usuario root del sistema. Por eso se entra con `sudo`:

```
$ sudo mysql
mysql> SELECT VERSION();
mysql> SHOW DATABASES;
mysql> exit
```

`SHOW DATABASES` muestra las bases de datos del sistema: `mysql` (usuarios y permisos), `information_schema`, `performance_schema` y `sys`. No se deben tocar a mano.

6.3. Asegurar la instalación

MySQL incluye un asistente que aplica las medidas básicas de seguridad:

```
sudo mysql_secure_installation
```

Pregunta, entre otras cosas, si se quiere activar la validación de la fortaleza de las contraseñas, y propone **eliminar los usuarios anónimos**, **impedir que root se conecte desde otras máquinas** y **eliminar la base de datos de prueba**. A todo ello hay que responder que sí. Según la versión, puede no pedir contraseña para root, porque root ya entra por `auth_socket`.

6.4. Bases de datos y usuarios

Una aplicación web **nunca debe conectarse a MySQL como root**. Cada aplicación tiene su base de datos y su propio usuario, con permisos solo sobre esa base de datos y solo los que necesita. Si alguien consiguiera las credenciales de la aplicación, el daño quedaría limitado a sus datos.

```
CREATE DATABASE clinica CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'clinica_app'@'localhost' IDENTIFIED BY 'Una-Contraseña-Segura-1';
GRANT SELECT, INSERT, UPDATE, DELETE ON clinica.* TO 'clinica_app'@'localhost';
SHOW GRANTS FOR 'clinica_app'@'localhost';
```

Algunas claves:

- Un usuario de MySQL se identifica por su nombre **y por el equipo desde el que se conecta**: `'clinica_app'@'localhost'` solo puede conectarse desde el propio servidor, que es lo que queremos cuando la aplicación y la base de datos están en la misma máquina.
- `utf8mb4` es la codificación que admite todos los caracteres, incluidos tildes, ñes y emojis.
- `GRANT` concede permisos y `REVOKE` los retira. Los permisos de la línea anterior permiten trabajar con los datos, pero no crear ni borrar tablas.

Para comprobar el acceso con el nuevo usuario:

```
mysql -u clinica_app -p clinica
```

6.5. Configuración básica

La configuración del servidor está en `/etc/mysql/mysql.conf.d/mysqld.cnf`. Las opciones más importantes para empezar son:

Opción	Qué hace
<code>bind-address = 127.0.0.1</code>	La interfaz en la que escucha MySQL. Con <code>127.0.0.1</code> solo acepta conexiones del propio servidor. Es el valor por defecto en Ubuntu, y el adecuado si la aplicación está en la misma máquina
<code>port = 3306</code>	El puerto en el que escucha

Tras cambiar la configuración hay que reiniciar el servicio con `sudo systemctl restart mysql`. Para comprobar en qué dirección y puerto escucha:

```
$ sudo ss -tlnp | grep mysql
LISTEN 0 151 127.0.0.1:3306 0.0.0.0:* users:((("mysqld",...))
```

6.6. Motores de almacenamiento

MySQL guarda las tablas mediante **motores de almacenamiento**. El motor por defecto es **InnoDB**, que admite **transacciones** (un conjunto de operaciones que se hace entero o no se hace) y **claves foráneas** (que mantienen la coherencia entre tablas relacionadas). Es el adecuado para casi cualquier aplicación. En versiones antiguas el motor por defecto era **MyISAM**, más simple y sin transacciones; aún puede aparecer en documentación desfasada.

6.7. Herramientas gráficas

Además de la terminal, hay herramientas gráficas para administrar MySQL: **MySQL Workbench**, una aplicación de escritorio, y **phpMyAdmin**, una aplicación web escrita en PHP que se instala en el propio servidor (apartado 7.5).

7. PHP y la integración de las piezas

7.1. Instalación

```
sudo apt install php libapache2-mod-php php-mysql
```

- `php` instala el intérprete de PHP.
- `libapache2-mod-php` instala el módulo de Apache que ejecuta PHP.
- `php-mysql` instala las extensiones para conectar con MySQL (**mysqli** y **PDO_MySQL**).

Para que Apache cargue el módulo y las extensiones nuevas hay que reiniciarlo:

```
sudo systemctl restart apache2
```

Y para comprobar la versión desde la terminal:

```
php -v
```

7.2. Cómo ejecuta Apache el código PHP

Cuando llega una petición de un fichero `.php`, Apache no lo envía tal cual: se lo pasa al intérprete de PHP, que ejecuta el código y devuelve el HTML resultante, y ese HTML es lo que Apache envía al navegador. Hay dos formas de conectar Apache con PHP:

Forma	Cómo funciona
Módulo de Apache (<code>mod_php</code> , paquete <code>libapache2-mod-php</code>)	PHP funciona dentro de los propios procesos de Apache. Es la forma más sencilla de montar, y la que usamos en el módulo
PHP-FPM	PHP funciona como un servicio independiente, y Apache (o Nginx) le pasa las peticiones. Es más eficiente con mucho tráfico y permite usar distintas versiones de PHP, por lo que es habitual en producción

Un detalle de funcionamiento. El módulo de PHP necesita que Apache trabaje con el modelo de procesos *prefork*. Al instalar `libapache2-mod-php`, Ubuntu hace el cambio automáticamente. Si se pasa después a PHP-FPM, hay que deshacerlo.

7.3. La configuración de PHP: `php.ini`

PHP se configura en el fichero **php.ini**. En Ubuntu hay uno distinto para cada forma de ejecución, en `/etc/php/<versión>/`:

Fichero	Se aplica a
<code>/etc/php/<versión>/apache2/php.ini</code>	El PHP que ejecuta Apache: el de las páginas web
<code>/etc/php/<versión>/cli/php.ini</code>	El PHP que se ejecuta en la terminal con la orden <code>php</code>

Es un error frecuente cambiar el de `cli` y esperar que afecte a la web. Algunas directivas importantes:

Directiva	Qué controla
<code>display_errors</code>	Si los errores se muestran en la página. Off en producción, porque revelan información interna; On solo en desarrollo
<code>error_log</code>	Dónde se guardan los errores
<code>upload_max_filesize</code> y <code>post_max_size</code>	El tamaño máximo de los ficheros que se pueden subir
<code>memory_limit</code>	La memoria máxima que puede usar un script
<code>max_execution_time</code>	El tiempo máximo de ejecución de un script

Tras modificar el `php.ini` de Apache hay que reiniciar Apache.

7.4. Extensiones

Muchas funciones de PHP vienen en **extensiones** que se instalan como paquetes independientes, con nombres del tipo `php-nombre`: `php-mysql` para MySQL, `php-gd` para imágenes, `php-curl` para peticiones a otros servidores, `php-mbstring` para textos con varios idiomas, `php-xml` o `php-zip`. Los gestores de contenidos que instalaremos más adelante piden varias de ellas. Para ver las extensiones cargadas:

```
php -m
```

7.5. phpMyAdmin

phpMyAdmin es una aplicación web, escrita en PHP, para administrar MySQL desde el navegador: crear bases de datos, tablas y usuarios, lanzar consultas o importar y exportar datos. Se instala con el paquete `phpmyadmin` y queda accesible en `http://servidor/phpmyadmin`.

Es muy cómoda, pero también es un objetivo habitual de ataques: da acceso a todas las bases de datos desde un navegador. En un servidor en producción, si se instala, debe protegerse (acceso solo desde determinadas IP, contraseñas robustas y HTTPS) o sustituirse por el acceso por terminal.

7.6. Comprobar que todo está integrado

PHP funciona en Apache. Se crea una página de prueba:

```
echo '<?php phpinfo();' | sudo tee /var/www/html/info.php
```

Al abrir `http://servidor/info.php` aparece una página con la versión de PHP, la ruta del `php.ini` cargado y la lista de extensiones. Si en lugar de eso se ve el código o se descarga el fichero, falta el módulo de PHP.

Borra `info.php` en cuanto termines. `phpinfo()` muestra muchísima información interna del servidor. Dejarla publicada es regalársela a cualquiera: `sudo rm /var/www/html/info.php`.

PHP conecta con MySQL. Un script mínimo con PDO, usando el usuario de aplicación del apartado 6.4:

```
<?php
try {
    $pdo = new PDO('mysql:host=localhost;dbname=clinica;charset=utf8mb4',
                  'clinica_app', 'Una-Contraseña-Segura-1');
    echo 'Conexión correcta. MySQL ' . $pdo->query('SELECT VERSION()')->fetchColumn();
} catch (PDOException $e) {
    echo 'Error de conexión: ' . $e->getMessage();
}
```

Si responde *Conexión correcta*, las cuatro piezas están integradas: Apache recibe la petición, PHP ejecuta el script y MySQL responde a la consulta.

8. Seguridad en los accesos al servidor

Un servidor web está expuesto por definición: su trabajo es atender peticiones de cualquiera. Por eso la seguridad no se añade al final, sino que se tiene en cuenta en cada paso de la instalación.

8.1. Tres principios

- **Mínimo privilegio.** Cada usuario, programa y servicio tiene solo los permisos que necesita para su trabajo, ni uno más. Si algo falla o alguien lo aprovecha, el daño queda limitado.
- **Reducir la superficie de ataque.** Todo lo que está instalado, abierto o publicado y no se usa es una puerta más. Se desactiva o se elimina.
- **Defensa en profundidad.** Ninguna medida es infalible, así que se combinan varias capas: si una falla, las demás siguen protegiendo.

8.2. Usuarios y permisos del sistema

- Se trabaja con un usuario normal y se usa **sudo** solo para las tareas de administración.
- Apache se ejecuta como **www-data**, sin privilegios. Ese usuario debe poder **leer** los ficheros de la web, pero no modificarlos, salvo en las carpetas concretas donde la aplicación necesite escribir (por ejemplo, la de ficheros subidos).
- Los permisos habituales para el contenido publicado son **755** para las carpetas y **644** para los ficheros: el propietario lee y escribe; el resto, solo lee.

```
sudo find /var/www/html -type d -exec chmod 755 {} \;  
sudo find /var/www/html -type f -exec chmod 644 {} \;
```

Nunca `chmod 777`. Dar todos los permisos a todo el mundo «para que funcione» permite a cualquier usuario o proceso del sistema modificar la web. Si una aplicación necesita escribir en una carpeta, se le da la propiedad de esa carpeta a `www-data`, y solo de esa.

8.3. Apache: no dar información de más

Por defecto, Apache en Ubuntu informa de su versión y de su sistema operativo en la cabecera `Server` y al pie de las páginas de error. Esa información ayuda a un atacante a buscar fallos conocidos de esa versión concreta. Se reduce en `/etc/apache2/conf-available/security.conf`:

```
ServerTokens Prod  
ServerSignature Off
```

Con `ServerTokens Prod` la cabecera queda en `Server: Apache`, sin versión ni sistema.

Otra fuga habitual es el **listado de directorios**: si una carpeta no tiene fichero índice (`index.html` o `index.php`), Apache muestra la lista de todo lo que contiene. Se desactiva quitando `Indexes` de las opciones del directorio, en `/etc/apache2/apache2.conf`:

```
<Directory /var/www/>
    Options -Indexes +FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Y siempre, tras cambiar la configuración, `sudo apache2ctl configtest` y `sudo systemctl reload apache2`. Se comprueba con `curl -I localhost` que la cabecera `Server` ya no muestra la versión.

8.4. MySQL: nadie entra por donde no debe

- **mysql_secure_installation** elimina los usuarios anónimos y la base de datos de prueba, e impide que root se conecte desde otras máquinas (apartado 6.3).
- **Un usuario por aplicación**, con permisos solo sobre su base de datos. Nunca root en una aplicación (apartado 6.4).
- **Sin acceso remoto** si no hace falta: con `bind-address = 127.0.0.1`, MySQL no es accesible desde la red, aunque alguien conozca un usuario y una contraseña.

8.5. El cortafuegos: ufw

Un **cortafuegos** decide qué conexiones de red se admiten. Ubuntu incluye **ufw** (*Uncomplicated Firewall*), inactivo por defecto. La política razonable para un servidor web es denegar todo lo que entra salvo lo necesario:

```
sudo ufw allow OpenSSH
sudo ufw allow "Apache Full"
sudo ufw enable
sudo ufw status verbose
```

`Apache Full` abre los puertos 80 y 443. MySQL (3306) no se abre: solo lo usa el propio servidor.

Primero SSH, después activar. Si administras el servidor por SSH y activas el cortafuegos sin haber permitido OpenSSH, te quedas fuera de tu propio servidor.

8.6. SSH

SSH permite administrar el servidor a distancia con una conexión cifrada. Escucha en el **puerto 22**, y es una de las puertas más atacadas de cualquier servidor en internet. Las medidas básicas son:

- **No permitir la entrada directa como root.** En Ubuntu, root no tiene contraseña, así que no puede entrar con contraseña por SSH.

- **Usar claves en lugar de contraseñas.** Se genera un par de claves en el equipo cliente (`ssh-keygen`), se copia la pública al servidor (`ssh-copy-id usuario@servidor`) y, una vez comprobado que se entra con la clave, se desactiva la autenticación por contraseña (`PasswordAuthentication no` en la configuración de SSH).
- **Mantenerlo actualizado** y limitar quién puede entrar.

8.7. HTTPS

Todo lo que viaja por HTTP va sin cifrar: contraseñas incluidas. Un servidor que se va a usar de verdad sirve sus páginas por **HTTPS**, con un **certificado digital** que acredita su identidad:

- Un **certificado autofirmado** lo genera el propio servidor. Cifra la conexión, pero el navegador avisa de que no puede confirmar la identidad del sitio. Vale para pruebas y redes internas.
- Un certificado emitido por una **autoridad de certificación** es reconocido por los navegadores. **Let's Encrypt** los emite gratis y se renuevan automáticamente con la herramienta **certbot**, pero necesita un dominio público que apunte al servidor.

En Apache, HTTPS se activa con el módulo `ssl` (`sudo a2enmod ssl`) y un host virtual en el puerto 443 con las rutas del certificado y de su clave.

8.8. Actualizaciones

La mayoría de los ataques aprovechan fallos ya conocidos y corregidos en versiones más recientes. Mantener el sistema actualizado (`sudo apt update && sudo apt upgrade`) es probablemente la medida de seguridad más eficaz de todas. Ubuntu puede instalar solo las actualizaciones de seguridad con el paquete `unattended-upgrades`.

8.9. Verificar la seguridad

Una medida que no se comprueba no es una medida. Estas órdenes permiten verificar lo anterior:

Qué se comprueba	Cómo
Con qué usuario se ejecuta Apache	<code>ps aux grep apache2</code>
Los permisos de la web	<code>ls -l /var/www/html</code>
Que Apache no revela su versión	<code>curl -I localhost</code>
Que no hay listado de directorios	Pedir en el navegador una carpeta sin índice: debe dar 403
Que MySQL solo escucha en local	<code>sudo ss -tlnp grep 3306</code>
Los permisos de un usuario de MySQL	<code>SHOW GRANTS FOR 'usuario'@'localhost';</code>
Qué puertos están abiertos en el cortafuegos	<code>sudo ufw status verbose</code>

En el aula. La seguridad de tu servidor forma parte del proyecto de la unidad: cada medida se aplica **y se verifica**, y en el cuaderno se documentan las dos cosas, la configuración y la prueba de que funciona.

9. Comprobación del sistema

Con todas las piezas instaladas, se comprueba el sistema de abajo arriba, de modo que cada paso confirme que el anterior funciona. Si algo falla, así se sabe exactamente en qué pieza está el problema.

Orden	Qué se comprueba	Resultado esperado
1	Apache está en marcha: <code>systemctl status apache2</code>	<code>active (running)</code>
2	Apache escucha en el puerto 80: <code>sudo ss -tlnp grep :80</code>	Una línea con <code>apache2</code>
3	Apache responde en local: <code>curl -I localhost</code>	<code>HTTP/1.1 200 OK</code>
4	Apache responde desde la red: navegador del anfitrión, <code>http://IP-del-servidor</code>	La página por defecto, <i>It works!</i>
5	PHP funciona en Apache: página con <code>phpinfo()</code>	La página de información de PHP
6	MySQL está en marcha: <code>systemctl status mysql</code>	<code>active (running)</code>
7	MySQL responde: <code>sudo mysql -e "SELECT VERSION();" </code>	La versión de MySQL
8	PHP conecta con MySQL: script de conexión con PDO	<i>Conexión correcta</i>

9.1. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
Apache no arranca	Error en la configuración, o puerto 80 ocupado por otro programa	<code>sudo apache2ctl configtest</code> y <code>sudo ss -tlnp grep :80</code> . Revisar <code>journalctl -u apache2</code>
El navegador del anfitrión no llega, pero <code>curl localhost</code> sí	La máquina virtual está en NAT, o hay un cortafuegos	Comprobar la IP con <code>ip a</code> (en NAT es <code>10.0.2.15</code>) y el modo de red; <code>sudo ufw status</code>
El <code>.php</code> se descarga o se ve el código	Falta el módulo de PHP para Apache	<code>sudo apt install libapache2-mod-php</code> y reiniciar Apache
403 Forbidden	Permisos insuficientes, o una carpeta sin índice con el listado desactivado	Revisar permisos con <code>ls -l</code> y que exista <code>index.php</code> o <code>index.html</code>
500 Internal Server Error	Un error en el código PHP o en un <code>.htaccess</code>	<code>sudo tail /var/log/apache2/error.log</code>
<i>could not find driver</i>	Falta la extensión de PHP para MySQL	<code>sudo apt install php-mysql</code> y reiniciar Apache
<i>Access denied for user</i>	Usuario, contraseña o equipo de origen incorrectos, o sin permisos sobre la base de datos	Revisar el usuario y sus permisos con <code>SHOW GRANTS</code>
<i>Can't connect to MySQL server</i>	MySQL está parado, o escucha en otra dirección	<code>systemctl status mysql</code> y revisar <code>bind-address</code>

Síntoma	Causa probable	Qué hacer
El ping a 8.8.8.8 falla con IP correcta	La red no sale a internet: proxy o cortafuegos de la red	Consultar la configuración de red del centro

Los registros dicen casi siempre qué pasa. Ante un error, antes de cambiar nada al azar, mira `/var/log/apache2/error.log` y `journalctl -u` del servicio. Un error leído es medio error resuelto.

10. Plataformas integradas

10.1. Qué son

Montar un servidor pieza a pieza, como en los apartados anteriores, lleva tiempo. Para trabajar en local existen **plataformas integradas**: paquetes que instalan de una vez un servidor web, un SGBD y un lenguaje de servidor ya configurados para funcionar juntos, con un panel de control para arrancarlos y pararlos.

Plataforma	Sistemas	Qué incluye
XAMPP	Windows, Linux y macOS	Apache, MariaDB, PHP y Perl, además de phpMyAdmin
WampServer	Windows	Apache, MySQL o MariaDB y PHP
MAMP	macOS y Windows	Apache o Nginx, MySQL y PHP
Laragon	Windows	Apache o Nginx, MySQL y PHP, con varias versiones

Una alternativa actual son los **contenedores**, con herramientas como **Docker**: cada pieza funciona en su propio contenedor y toda la pila se describe en un fichero, de modo que se puede reproducir igual en cualquier equipo.

10.2. Ventajas e inconvenientes

Sus **ventajas** son evidentes en un entorno de desarrollo o de pruebas: se instalan en minutos, funcionan en varios sistemas operativos, son gratuitas y permiten probar una aplicación o una nueva versión en el propio ordenador antes de llevarla al servidor real, sin necesidad de conectarse a él ni de transferir ficheros cada vez.

Sus **inconvenientes** los hacen inadecuados para producción: vienen con una configuración pensada para la comodidad, no para la seguridad (usuarios sin contraseña, herramientas de administración abiertas), y no se integran con el sistema de actualizaciones del sistema operativo.

Para probar, no para publicar. Una plataforma integrada es un banco de pruebas. Lo importante al elegirla es que sus versiones de PHP y del SGBD sean lo más parecidas posible a las del servidor real: así lo que funciona en local funcionará también allí.

10.3. XAMPP

XAMPP se descarga desde la web de **Apache Friends**. Lo instala todo en una carpeta (C:\xampp en Windows, /opt/lampp en Linux), con la raíz de la web en su subcarpeta `htdocs`. Desde el **panel de control** se arrancan y detienen Apache y MySQL (que en XAMPP es MariaDB), y se accede a phpMyAdmin en `http://localhost/phpmyadmin`.

Conflicto de puertos. Si en el equipo ya hay otro servidor web o de bases de datos funcionando, XAMPP no podrá arrancar el suyo en el mismo puerto (80 para la web, 3306 para la base de datos). Hay que detener el otro servicio o cambiar el puerto de XAMPP en su configuración.

10.4. La instalación manual en Windows

Montar las piezas por separado en Windows también es posible, aunque es menos habitual para servidores:

- **Apache:** la Apache Software Foundation no distribuye binarios para Windows, sino que remite a distribuciones de terceros; la más conocida es **Apache Lounge**. Se descomprime, se configura en `conf\httpd.conf` y se instala como servicio de Windows.
- **PHP:** se descarga de la web oficial de PHP para Windows. Para usarlo como módulo de Apache se necesita la versión *Thread Safe*, y se carga desde la configuración de Apache.
- **MySQL:** tiene un instalador para Windows que lo configura como servicio.

En Windows, además, los servicios se gestionan desde la consola de servicios y el cortafuegos de Windows puede bloquear los puertos.

11. Documentación de la instalación

11.1. Por qué documentar

Un servidor bien documentado se puede **reinstalar** siguiendo los mismos pasos, **mantener** por otra persona y **diagnosticar** cuando falla, porque se sabe qué había y qué se cambió. La documentación debe tratarse como una pieza más del servidor: se crea al instalar y **se actualiza con cada cambio**. Una documentación que no refleja el estado actual del servidor es casi peor que no tenerla.

11.2. Qué documentar

Apartado	Contenido
Identificación	Nombre del servidor, función, IP, responsable
Sistema	Sistema operativo y versión, recursos (CPU, RAM, disco)
Software	Cada pieza instalada con su versión exacta: servidor web, SGBD, PHP y sus extensiones
Configuración	Los cambios hechos respecto a la configuración por defecto, fichero a fichero, y por qué
Sitios y bases de datos	Cada sitio web, su carpeta y su dominio; cada base de datos, su usuario y sus permisos (nunca las contraseñas en claro)
Seguridad	Las medidas aplicadas y cómo se verificaron
Incidencias	Los problemas surgidos, su causa y cómo se resolvieron
Registro de cambios	Cada modificación posterior, con su fecha y quién la hizo

Las versiones se obtienen con estas órdenes:

```
lsb_release -a
apache2 -v
mysql --version
php -v
php -m
```

11.3. Cómo documentar

Para cada paso de una instalación, una buena documentación responde a cuatro preguntas: **qué** se ha hecho (la orden o la opción elegida), **por qué** se ha hecho así (la decisión y sus alternativas), **para qué** sirve (qué se consigue) y **cómo se ha comprobado** (la evidencia: una captura o la salida de una orden).

Las **incidencias** son la parte más valiosa: un problema documentado con su causa y su solución ahorra horas la próxima vez que aparezca, a ti o a quien venga detrás.

En el aula. Es el formato del cuaderno del proyecto: para cada paso, captura, qué he hecho, por qué y para qué sirve, y al final las incidencias. No se documenta para el profesor, sino para poder repetir y mantener el trabajo.

Ideas clave de la unidad

- Una aplicación web se ejecuta en parte en el **cliente** (HTML, CSS y JavaScript en el navegador) y en parte en el **servidor** (PHP y la base de datos). Todo lo que se comprueba en el cliente se puede saltar; la seguridad se garantiza en el servidor.
- **HTTP** es un protocolo de petición y respuesta. El código de estado resume el resultado: 2xx éxito, 3xx redirección, 4xx error del cliente, 5xx error del servidor. HTTPS es HTTP cifrado.
- Un servidor **LAMP** combina Linux, Apache, MySQL y PHP. En Ubuntu se instala con los paquetes `apache2`, `mysql-server`, `php`, `libapache2-mod-php` y `php-mysql`.
- Antes de instalar se **analizan los requisitos** y se **prepara el sistema**: actualizado, con la red comprobada y con los puertos libres.
- **Apache** se configura en `/etc/apache2/`, publica lo que hay en `/var/www/html/` y registra su actividad en `/var/log/apache2/`. Tras cada cambio: `configtest` y `reload`.
- **MySQL** se asegura con `mysql_secure_installation`, y cada aplicación usa su propio usuario con los permisos justos, nunca root.
- La **seguridad** se basa en el mínimo privilegio, en reducir la superficie de ataque y en combinar medidas; y cada medida **se aplica y se verifica**.
- El sistema se **comprueba de abajo arriba**, y ante un error se leen los registros antes de tocar nada.
- Las **plataformas integradas** como XAMPP sirven para probar en local, no para producción.
- Una instalación no está terminada hasta que está **documentada**.

Glosario

Término	Significado
Anfitrión (<i>host</i>)	El equipo físico sobre el que funcionan las máquinas virtuales
apt	El gestor de paquetes de Ubuntu
Cabecera HTTP	Dato adicional, en forma <code>nombre: valor</code> , de una petición o una respuesta
Certificado digital	Documento electrónico que acredita la identidad de un servidor en HTTPS
Cliente	El programa que pide un servicio; en la web, el navegador
Código de estado	Número de tres cifras con el que el servidor resume el resultado de una petición
Cortafuegos	Sistema que decide qué conexiones de red se permiten
DHCP	Protocolo que asigna automáticamente la configuración de red a los equipos
DNS	Sistema que traduce nombres de dominio a direcciones IP
DocumentRoot	La carpeta cuyo contenido publica un sitio web
Hipervisor	Software que crea y ejecuta máquinas virtuales
Host virtual	Configuración que permite alojar varios sitios web en un mismo servidor
Hostname	El nombre de un equipo en la red
HTTP / HTTPS	Protocolo de comunicación de la web / su versión cifrada
LAMP	Pila formada por Linux, Apache, MySQL y PHP
LTS	Versión con soporte de larga duración
Módulo de Apache	Componente que añade una funcionalidad al servidor web
Página dinámica	Página que el servidor genera en el momento de cada petición
Página estática	Fichero que el servidor envía tal cual
Paquete	Software preparado para instalarse con el gestor de paquetes
php.ini	El fichero de configuración de PHP
Puerto	Número que identifica un servicio dentro de un equipo: 80 HTTP, 443 HTTPS, 22 SSH, 3306 MySQL
Repositorio	Servidor del que se descargan los paquetes
Servidor	El programa que presta un servicio, o la máquina en la que se ejecuta
SGBD	Sistema gestor de bases de datos
SSH	Protocolo para administrar un equipo a distancia de forma cifrada
sudo	Orden que permite ejecutar tareas de administración a un usuario autorizado
www-data	El usuario sin privilegios con el que se ejecuta Apache en Ubuntu

Autoevaluación

1. Un usuario modifica en su navegador el precio de un producto antes de pagar, y la tienda le cobra ese precio. ¿Qué principio ha incumplido la aplicación?
 - a) Los cálculos deben hacerse en el cliente para no cargar el servidor.
 - b) Los datos que importan deben comprobarse en el servidor.
 - c) HTTP no permite enviar precios.
2. Un servidor responde a una petición con un código 503. ¿Dónde está el problema?
 - a) En el cliente, que ha pedido un recurso que no existe.
 - b) En el servidor, que no puede atender la petición.
 - c) En ninguno: es una redirección.
3. ¿Qué hace la orden `curl -I`?
 - a) Instala curl.
 - b) Hace una petición y muestra solo las cabeceras de la respuesta.
 - c) Muestra la IP del servidor.
4. Al abrir un fichero `.php` en el navegador, se descarga en lugar de mostrarse. ¿Qué falta con más probabilidad?
 - a) php-mysql
 - b) libapache2-mod-php
 - c) mysql-server
5. ¿Qué modo de red de VirtualBox da a la máquina virtual su propia IP en la red local?
 - a) NAT
 - b) Solo anfitrión
 - c) Adaptador puente
6. ¿Qué diferencia hay entre `apt update` y `apt upgrade`?
 - a) Ninguna: son sinónimos.
 - b) update actualiza la lista de paquetes disponibles; upgrade instala las nuevas versiones.
 - c) update instala las nuevas versiones; upgrade actualiza la lista.
7. ¿Con qué usuario debe conectarse una aplicación web a su base de datos?
 - a) Con root, para no tener problemas de permisos.
 - b) Con un usuario propio, con permisos solo sobre su base de datos.
 - c) Con el usuario anónimo.
8. ¿Qué consigue la directiva `ServerTokens Prod` en Apache?
 - a) Que la cabecera Server no muestre la versión ni el sistema operativo.
 - b) Que Apache funcione en modo producción, más rápido.
 - c) Que se desactive el listado de directorios.
9. ¿Para qué es adecuada una plataforma integrada como XAMPP?
 - a) Para un servidor en producción, porque es más fácil de mantener.
 - b) Para desarrollo y pruebas en local.
 - c) Solo para Windows.

10. ¿Cuál de estos datos **no** debe figurar en claro en la documentación de un servidor?

- a) La versión de PHP.
- b) Las contraseñas de los usuarios de la base de datos.
- c) Las incidencias surgidas durante la instalación.

Soluciones: 1 b · 2 b · 3 b · 4 b · 5 c · 6 b · 7 b · 8 a · 9 b · 10 b